Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (Raise Hand).
When instructor calls on you, click microphone icon to unmute. When you've finished speaking, *be sure to mute yourself again*.

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class for their own private use. Releasing these recordings is strictly prohibited.

THE GEORGE WASHINGTON UNIVERSITY
WASHINGTON, DC

SEAS 8515

Data Engineering for AI

Professor Steven Cocke

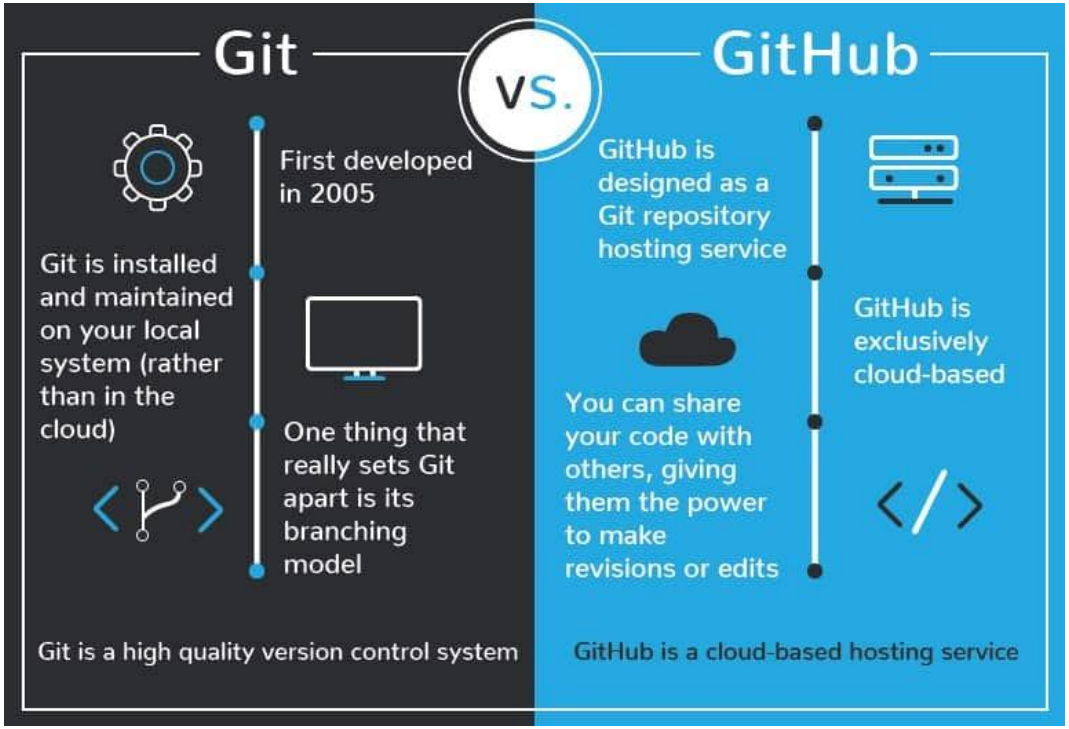
Week 2: May 23, 2026

Agenda

- Continuing Python Foundations I
- Python Foundations II
  - Installing Libraries, Importing Packages, Scripts
  - Common Libraries
  - Reading and Writing Data
  - Data Viz
  - Basic Git Usage
- Assignment Overview for This Week

What is Git?

- Git is a distributed version control system
- Tracks changes to files over time
- Enables reproducibility, collaboration, rollback, and much more
- GitHub is the what hosts Git



https://medium.com/nerd-for-tech/getting-started-with-git-and-github-2b9a86df6d9f

Why Git Matters

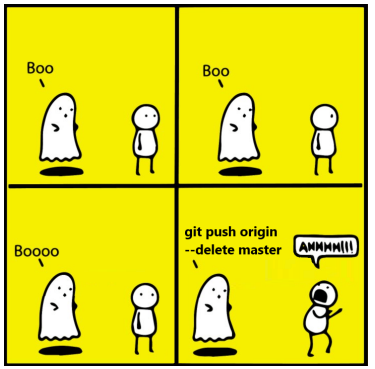
- Reproduce experiments from N months ago
- Compare model variants without copy-pasting folders
- Collaborate without overwriting each other's work
- Support for peer review and publication
- Allows for rapid development
- Software development is explicit and auditable

Source

Source

Source

Source



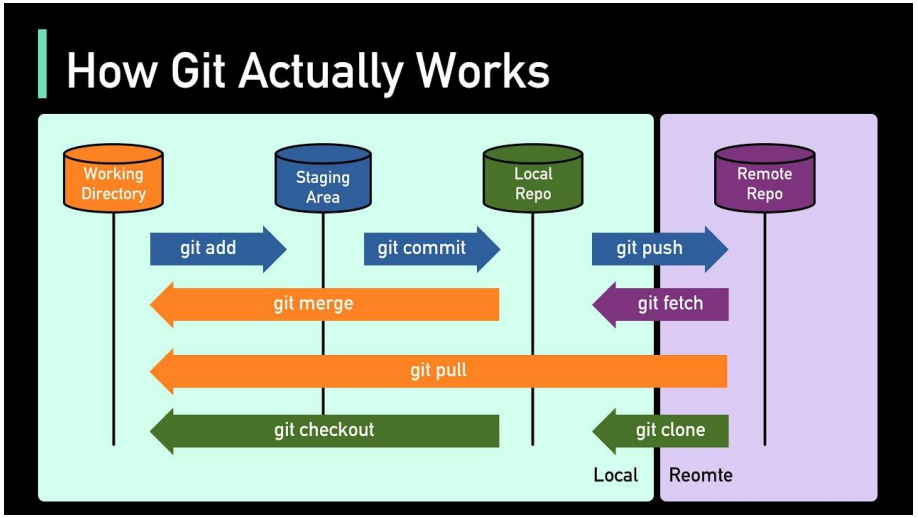
Source

If your work isn't version-controlled, it isn't reproducible.

THE GEORGE WASHINGTON UNIVERSITY
WASHINGTON, DC

Core Git Concepts/Terminology

Concept	Meaning
Repository	Project + history
Commit	Snapshot of changes
Branch	Independent line of development
Remote	Shared repository (e.g., GitHub)
Working tree	Files on disk
Staging area	What will be committed



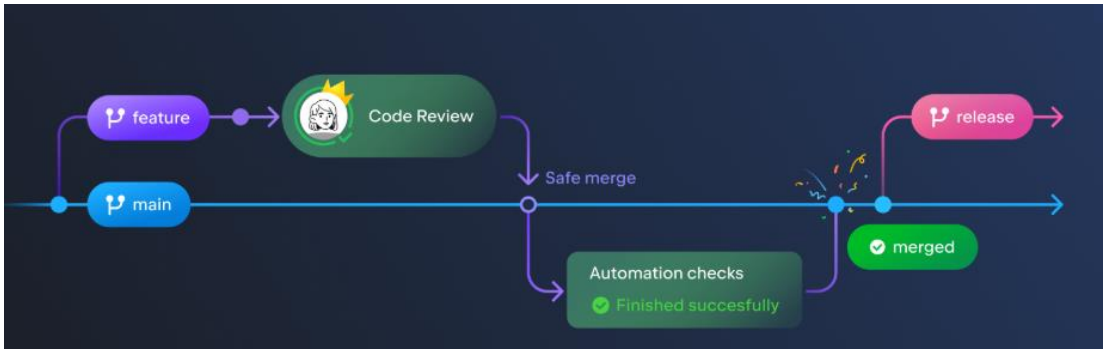
Working Directory -> Staging -> Commit -> History

Most Common Git Commands

Command	Workflow Category	Description
`git init`	Local	Initializes a new Git repository in the current directory
`git status`	Local	Shows the current state of the working directory and staging area
`git add <file>`	Local	Stages file changes to be included in the next commit
`git add .`	Local	Stages all modified and new files
`git commit -m "message"`	Local	Records a snapshot of staged changes with a descriptive message
`git log`	Local	Displays the commit history
`git log --oneline`	Local	Displays a compact, readable commit history
`git clone <url>`	Remote	Creates a local copy of an existing remote repository
`git pull`	Remote	Fetches and merges changes from the remote repository
`git push`	Remote	Sends local commits to the remote repository
`git branch`	Branching	Lists all local branches
`git checkout <branch>`	Branching	Switches to an existing branch
`git checkout -b <branch>`	Branching	Creates and switches to a new branch
`git merge <branch>`	Branching	Merges changes from one branch into the current branch

Typical Git Workflow

1. Pull latest changes
2. Create or switch branch
3. Make changes
4. Commit frequently
5. Push
6. Open pull request (PR)
7. Review & merge

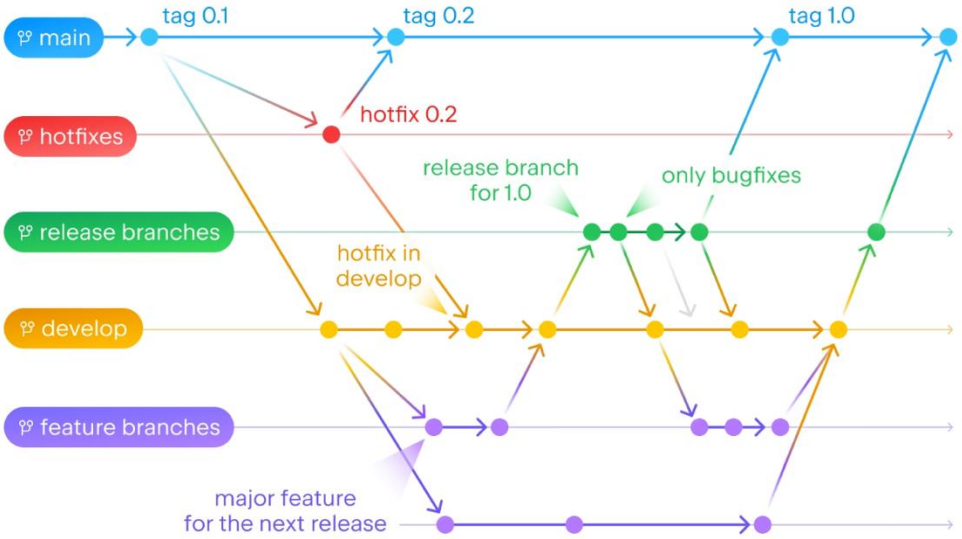


https://blog.jetbrains.com/space/2023/04/18/space-git-flow/

Small, frequent, meaningful commits > giant commits

Branching Strategy

- The purpose of branching is to
  - Prevent chaos
  - Organize development into specific features
- Typically:
  - The main branch is not directly committed to
  - Feature/* branches or a similar naming convention is used for the organization of branches



https://blog.jetbrains.com/space/2023/04/18/space-git-flow/

```
main
  ├── feature/data-loader
  ├── feature/model-v1
  └── feature/evaluation
```

How to Structure a Python Repository

- Every organization typically has it's own standards for repository structure, but commonly:
  - Repositories should be "clean"/organized
  - No large datasets in Git
    - Often data/ is a placeholder for local work
  - Scripts live in src
  - A descriptive README.md that explains not only what is in the repository but how it is intended to be used
  - No private keys, tokens, etc
  - Leverage .gitignore to avoid committing:
    - Large datasets, credentials, model checkpoints, virtual environments, and generated files (large logs)

```
project-name/
├── README.md
├── requirements.txt
├── .gitignore
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
├── src/
│   └── project_name/
│       ├── __init__.py
│       ├── data.py
│       ├── model.py
│       └── train.py
├── tests/
└── experiments/
```

Jupyter Notebooks and Git

- Notebooks store outputs and execution state
- Diffs and merge conflicts are hard to read
- Use notebooks for exploration, not production code
- Move stable logic into .py files
- Clear outputs before committing

Best Practices

- Don’t work directly on main
- Commit early and often (small, focused commits)
- Never commit data, credentials, or generated artifacts
- Use branches for all work
- Git history is a safety net
  - Most mistakes are recoverable

Git is forgiving; not committing is the real risk.

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

What to work on this week

What To Work On This Week

- Homework #2 (Hands-on Assignment)
- Finish review of first two notebooks
- Reading: Chapters 1-3 of Pro Git: https://git-scm.com/book/en/v2
  - Ch1: Getting Started
  - Ch 2: Git Basics
  - Ch 3: Git Branching
  - **I have also uploaded the PDF to Additional Readings folder on Blackboard**
- Take this time to review Python and Git concepts, attend office hours or email me if help is needed.